ProEm: Protocol Emulator ASIC

Design Reference

Paul Vincent Bezzina

26 September 2026

Process	IHP SG13G2 130 nm, through Tiny Tapeout (6 × 4 tiles)
System clock	50 MHz (60 MHz for 10BASE-T transmit)
Channels	2 independent protocol channels, A and B
Host link	UART in (auto-baud), UART out (rate set by `LINK`)
Standards verified	UART, SPI (controller and target), I^2C (controller), PS/2, 10BASE-T receive, Manchester transmit, USB low speed receive and transmit
Logic	12,031 standard cells (≈ 50 % of the tile budget)
Timing	6.6 ns worst path at 1.08 V, 125 °C (20 ns budget at 50 MHz)
Instruction set	ISA v0.4 (separate document: *ProEm Instruction Set*)

Contents

1 Overview **2**

2 Top-level interface **4**

3 Blocks **4**

3.1 UART receiver and auto-baud (`uart_rx`) . . . 4

3.2 Control unit (`control_unit`) . . . 4

3.3 Frame chain (`frame_chain`) . . . 5

3.4 Host FIFO and entry dispatcher . . . 5

3.5 Line codec (`line_codec`) . . . 5

3.6 Return path (`return_path`, `uart_tx`) . . . 6

4 Host protocol **7**

4.1 Start-up . . . 7

4.2 Return messages . . . 7

4.3 Choosing `RUN` modes . . . 7

5 Protocol recipes **7**

6 Verification **9**

7 Implementation results **9**

8 Limits and future work **10**

9 Files **11**

1 Overview

ProEm is an open-source protocol emulator ASIC. The chip lets an RP2040 speak many serial protocols through one piece of hardware. The RP2040 does not bit-bang anything. It describes a frame to the chip in a few instruction bytes: how the frame starts, which fields follow, which way each field goes, how the frame ends, how the line is coded and where the bit clock comes from. The chip then runs the frame at up to one bit per system clock, with exact timing, and sends back what it received together with a status for every frame.

The design has four parts:

1. **Host side.** A UART receiver with auto-baud feeds the control unit, which decodes ISA v0.4 instructions into registered configuration writes. A return path sends received data, frame status and link events back on a second UART.
2. **Input pipeline.** Every input pin passes through two synchroniser flip-flops and one more register before any logic uses it.
3. **Two channels.** Each channel has a host FIFO for write data, a *frame chain* that runs the frame, an *entry dispatcher*, and a *line codec* that turns pins into bits and ticks, and bits into pin levels.
4. **Outputs.** Each channel drives a data output, a pair output (the complement, for USB D−), a clock output and a select output, each with an output enable. Bidirectional pads are read back through the input pins.

The two channels run at the same time. That is how the chip does SPI full duplex (one channel writes MOSI, the other reads MISO), UART receive and transmit together, or a transmitter looped into a receiver for self-test.

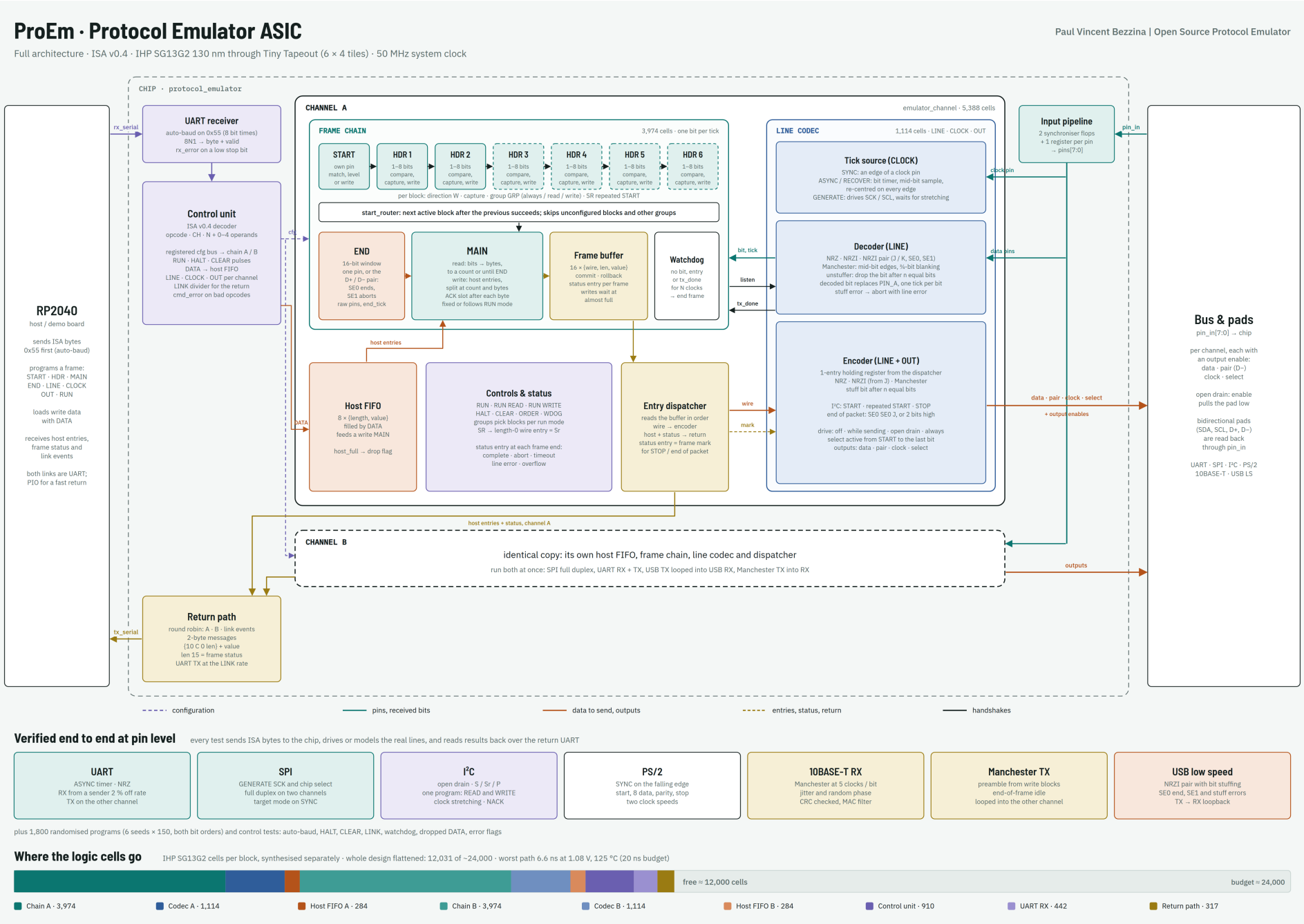


Figure 1: The whole chip. Channel A is shown in full; channel B is an identical copy.

2 Top-level interface

The top module is `protocol_emulator`. Its ports are generic; the Tiny Tapeout wrapper maps them onto `ui_in`, `uo_out` and the bidirectional `uio` pins.

Port	Dir	Meaning
`clk`, `rst_n`	in	System clock and active-low asynchronous reset
`rx_serial`	in	Instruction bytes from the RP2040, 8N1. The first byte after reset must be `0x55`
`pin_in[7:0]`	in	Protocol input pins, shared by both channels
`tx_serial`	out	Return UART to the RP2040
`data_out_x`, `data_oe_x`	out	Channel x data output and output enable. In open-drain mode `out` is 0 and `oe` pulls the pad low
`pair_out_x`, `pair_oe_x`	out	Complement of the data output (USB D−)
`clock_out_x`, `clock_oe_x`	out	Generated clock (SCK, SCL)
`select_out_x`	out	Chip select: active from the frame's START until its last bit has gone out
`armed_x`, `host_full_x`, `baud_locked`	out	Debug status

Pads. A bidirectional line such as SDA is a pad driven by `data_out`/`data_oe` and read back on one of the `pin_in` bits. I^2C uses open drain on both SDA and SCL, so the pads need external pull-ups. USB low speed needs 1.5 kΩ from D− to 3.3 V when the chip is the device, and the pad voltage must reach 2.8 V or a transceiver is needed.

Input latency. The pipeline adds three clocks between a pad and the logic. Every pin is delayed equally, so a data pin and a clock pin stay aligned.

3 Blocks

3.1 UART receiver and auto-baud (`uart_rx`)

After reset the receiver waits for the first falling edge, then counts clocks until the fifth falling edge. For `0x55` those five edges are exactly 8 bit times apart, so the clocks per bit are the count divided by 8. The receiver then waits for the stop bit and becomes a normal 8N1 receiver at that rate. The sync byte is not passed on. A byte whose stop bit is low raises `rx_error`. The measured rate is also the default rate of the return UART.

3.2 Control unit (`control_unit`)

The control unit reads an opcode byte {`OP, CH, N`}, collects 0–4 operand bytes, and in one further clock turns the instruction into:

- a write on the shared configuration bus with a write enable for channel A or B (`START`, `HDR`, `MAIN`, `END`, `WDOG`, `ORDER`);
- a pulse: `RUN` with its mode, `HALT` or `CLEAR`;
- a host FIFO write (`DATA`);
- a register update: `LINE`, `CLOCK` and `OUT` per channel, `LINK` for the return link.

Every output is a register, so the configuration bus forms a pipeline stage and the decode logic is never in a channel's timing path. `START` patterns are given as plain numbers; the control unit aligns them for the channel's bit order. Reserved opcodes and `HDR 0`/`HDR 7` raise `cmd_error`.

3.3 Frame chain (`frame_chain`)

The frame chain is the frame engine: one START block, up to six header blocks and one MAIN block, run strictly in order, one bit per tick.

- **START** slides an 8-bit window over its own pin and matches a pattern of 1–8 bits; with 0 bits it waits for a level (SPI chip select); as a write block it sends its value when `RUN` starts the frame.
- **Headers** each compare, capture or send a 1–8 bit field. Each has a group (always, read runs, write runs) and an optional repeated START before it.
- **start_router** enables the next active block after the previous one succeeds. It skips unconfigured blocks and blocks whose group is not in the current run mode.
- **MAIN** reads bits into bytes, to a count or until END; or writes host entries, splitting them at the frame's bit count and, with the ACK slot on, at byte edges. After each byte it can add one bit the other way (I^2C ACK). Its direction is fixed or follows the run mode.
- **END** slides a 16-bit window over one raw pin, or over the D+/D− pair compare: both low (SE0) is a 1, both high (SE1) aborts with a line error. After an abort, START waits until END has matched again, so it never searches inside the rest of a rejected frame.
- **Watchdog** ends a frame when no bit, host entry or sent entry is seen for a set number of clocks.
- **Frame buffer** holds 16 entries {`wire, length, value`}. Written values are released at once; captured values are released when MAIN starts and dropped on abort. At the end of every frame the chain adds a *status entry* (length 15) with complete, abort, timeout, line error and overflow flags.

Read blocks only take ticks once every bit written ahead of them has left the chip (`line_free`), so a frame can send a request and then read the reply. Write blocks wait when the buffer is almost full, so written bits are never lost; received entries that arrive with the buffer full are dropped and the frame's status has the overflow flag.

3.4 Host FIFO and entry dispatcher

The host FIFO holds 8 {`length, value`} entries loaded by `DATA` and feeds a write MAIN. A `DATA` arriving while it is full is dropped and reported as a link event.

The entry dispatcher reads the frame buffer in order. Wire entries go to the codec's encoder; host entries and status entries go to the return path. When a status entry reaches the head, every bit of that frame has already been handed to the encoder, so the dispatcher gives the encoder a *frame mark* at that moment. The encoder uses it to send the I^2C STOP or the end of packet at exactly the right place.

3.5 Line codec (`line_codec`)

The codec sits between the pins and the frame chain. `CLOCK` chooses the tick source, `LINE` the line code, `OUT` the output behaviour.

Tick source.

- **SYNC**: a tick on the rising, falling or both edges of a clock pin (SPI target, PS/2, a read-back clock).
- **ASYNC / RECOVER**: a bit timer of `DIV` clocks per bit that samples mid-bit and re-centres on every edge of the data line. The decoded bit replaces `LINE`'s `PIN_A` in the pins the chain sees, with one tick per bit.
- **GENERATE**: the codec drives the clock output, `DIV` clocks per half period, only while a bit is waiting to go out or a read block wants one. The sampling edge gives the tick. With open drain it waits for the clock read-back pin to go high, which gives I^{2}C clock stretching.

Decoder. NRZ takes the mid-bit sample. NRZI compares it with the previous one (same = 1). The NRZI pair code reads J/K from D+ and D− and gives no bits during SE0 or SE1. Manchester decodes on the mid-bit edges: the new level is the bit, and edges within $\frac{3}{4}$ of a bit after an accepted edge are ignored, which skips the boundary edges between equal bits. An unstuffer drops the bit after n equal bits; a stuffed bit with the wrong value aborts the frame with a line error.

Encoder. It holds one entry from the dispatcher and sends it bit 0 first (the chain has already reversed MSB-first fields). It encodes NRZ, NRZI starting from J, or Manchester, inserts a stuff bit after n equal bits, and continues seamlessly into the next entry. In I^{2}C mode it sends START before a transaction, repeated START for a length-0 entry, and STOP at the frame mark. With end of packet on, the frame mark produces two bit times of SE0 and one of J on a pair, or two bit times high on a single line. `tx_done` after each entry lets the chain know the line is free.

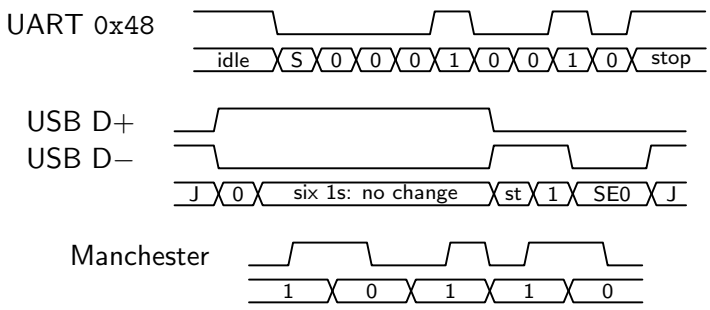


Figure 2: Three line codes from the same codec. UART: NRZ, LSB first, start and stop bits. USB low speed: NRZI on D+/D− (a 0 changes the line), a stuff bit (st) after six 1s, then the end of packet. 10BASE-T: Manchester, low-to-high in the middle of the bit = 1.

Outputs. `DRIVE` is off after reset, push-pull while sending, open drain, or push-pull always (with an idle level, for a UART line). The pair output carries the complement. The select output is active from the frame's START until its last bit has gone out.

3.6 Return path (`return_path`, `uart_tx`)

The return path takes host entries and status entries from both channels and link events, in round robin, and sends each as a two-byte message on the return UART at the `LINK` rate (or the auto-baud rate when `LINK` is 0).

4 Host protocol

4.1 Start-up

1. Release reset and hold `rx_serial` high for at least one byte time.
2. Send `0x55`. The chip measures the rate from it; `baud_locked` goes high.
3. Optionally send `LINK` to speed up the return link (`E0 00 04` = 4 clocks per bit, 12.5 Mbaud at 50 MHz).
4. Program each channel: `LINE`, `CLOCK`, `OUT`, `ORDER`, `WDOG`, the blocks, then `RUN`.

`CLEAR` returns a channel to its reset state, including its outputs; `HALT` stops a frame and disarms the channel but keeps its program.

4.2 Return messages

First byte	Second	Meaning
`10 C 0 LLLL`	value	Host entry from channel `C`, length 0–8. Length 0 with value `02` is a NACK from a writing MAIN
`10 C 0 1111`	flags	End of a frame on channel `C`: bit 0 complete, 1 abort, 2 timeout, 3 line error, 4 overflow
`11 000000`	flags	Link events: bit 0 `rx_error`, 1 `cmd_error`, 2 `DATA` dropped on A, 3 on B

A frame's host entries always come before its status message. A simple receiver on the RP2040:

```
void on_return_message(uint8_t head, uint8_t value) {
    if ((head & 0xC0) == 0xC0) { link_events(value); return; }
    int chan = (head >> 5) & 1, len = head & 0x0F;
    if (len == 15) frame_done(chan, value);        /* status flags   */
    else           frame_data(chan, len, value);   /* 0..8 bits      */
}
```

4.3 Choosing `RUN` modes

- **Receiving:** plain `RUN` arms the channel; it stays armed after every frame.
- **Packets** (USB, Ethernet, I^2C transactions): load the data with `DATA`, then `RUN WRITE` or `RUN READ`. The channel starts exactly one frame per `RUN`.
- **Streams** (UART transmit): plain `RUN`. The channel also starts a new frame whenever data is waiting.

5 Protocol recipes

Each recipe lists the instruction bytes. The bytes were produced by the test bench's own encoder, and the same programs (with smaller dividers, to keep simulation short) pass the tests in Section 6. Dividers are for a 50 MHz clock.

UART receive, 115 200 baud, channel A. RX on pin 0. `LINE` none on pin 0; `CLOCK` ASYNC, 434 clocks per bit; START reads the start bit; MAIN reads 8 data bits and the stop bit.

```
A0 00 00   B0 40 00 01 B2   10 01 00 00   30 00 00 09   70
```

UART transmit, channel B. Push-pull output, idle high; START writes the start bit; MAIN writes 9 bits. Each byte is two `DATA` entries: the byte and a stop bit.

```
A8 00 00   B8 40 00 01 B2   D8 E0   18 81 00 00   38 40 00 09
CF 48   C8 01   78                                  ("H", then RUN)
```

SPI controller, mode 0, MSB first, full duplex. Channel A generates SCK (half period 6 clocks) and chip select, writes MOSI from `DATA`. Channel B reads MISO on pin 3, clocked by SCK read back on pin 1, started by chip select read back on pin 2.

```
A: 61   A0 00 00   B0 81 00 00 06   D0 C0   50 0F A0   10 80 00 00   30 40 00 10
B: 69   A8 03 00   B8 01 00 00 00   18 00 02 00   38 03 00 10   78
transfer:   C7 A5   C7 5A   70
```

I²C controller, 100 kHz, register read and write in one program. Device 0x48, register 0x05. SDA read back on pin 0, SCL on pin 1; open drain with I²C conditions.

```
61   A0 00 00   B0 89 00 00 FA   D0 A4   50 EA 60
10 88 00 90   21 01 00 00   22 88 00 05   23 01 00 00
24 98 80 91   25 11 00 00   30 90 00 10
write:  C7 A1   C7 B2   72        bus: S 90 A 05 A A1 A B2 A P
read:   71                        bus: S 90 A 05 A Sr 91 A d0 A d1 N P
```

PS/2 keyboard to host, channel B. Clock on pin 4 (falling edge), data on pin 5. Each key gives {`8, code`} and {`2, parity + stop`}.

```
A8 05 00   B8 14 00 00 00   18 01 05 00   38 05 00 0A   78
```

10BASE-T receive, broadcast frames. Manchester line on pin 0; END sees the line high for 8 clocks at the end of the frame; return link at 4 clocks per bit.

```
E0 00 04   A0 60 00   B0 C0 40 00 05   10 08 00 D5
21 48 00 FF   22 48 00 FF   23 48 00 FF   24 48 00 FF   25 48 00 FF   26 48 00 FF
30 20 00 00   40 40 00 00 FF   70
```

Manchester transmit (6 clocks per bit, a 60 MHz clock for 10 Mb/s). The preamble comes from START and six write headers; MAIN sends up to 8 `DATA` bytes (SFD first); end of packet holds the line high for two bit times.

```
A0 60 00   B0 C0 00 00 06   D0 48   10 88 00 55
21 88 00 55   22 88 00 55   23 88 00 55   24 88 00 55   25 88 00 55   26 88 00 55
30 40 00 40      then DATA entries and RUN WRITE (72)
```

USB low speed receive. D+ on pin 0, D− on pin 1; NRZI pair with unstuffing; END on the pair, 2 bit times of SE0, SE1 aborts.

```
E0 00 04   A0 40 31   B0 C0 80 00 21   10 08 00 80   30 20 00 00
40 10 A1 00 03   70
```

USB low speed transmit, channel B. SYNC from START, PID `0x4B` from a header, 32 data bits from `DATA`, stuffing and end of packet.

```
A8 40 31   B8 C0 00 00 21   D8 58   18 88 00 80   29 88 00 4B   38 40 00 20
CF FF   CF FF   CF 7E   CF 00   7A
```

6 Verification

The test bench (`sim/bench.h`) simulates the whole chip with Yosys CXXRTL. Every test does what the RP2040 would do: it resets the chip, sends `0x55`, sends instruction bytes on `rx_serial`, and decodes the return UART on `tx_serial` into host entries, frame status and link events. Protocol lines are driven or modelled at the pins every clock: open-drain buses are wired-AND, clocks are looped back through `pin_in`, and devices are behavioural models.

Test	What it checks	Result
`t_control`	Auto-baud at 16 and 7 clocks per bit; read frames; header mismatch; watchdog timeout; `HALT`, re-`RUN`; `CLEAR`; START alignment in both bit orders; `LINK`; `cmd_error`, `rx_error`; `DATA` dropped on a full FIFO	Pass
`t_uart`	Channel A receives 5 bytes from a sender 2 % fast, then 2 % slow; channel B transmits "Hi!" decoded by a model	Pass
`t_spi`	Controller full duplex against a slave model; target mode against a master model; SCK half period 4, 6, 12 clocks	Pass
`t_i2c`	Device model on an open-drain bus: `RUN WRITE`, `RUN READ` with Sr, clock stretching, address NACK with STOP; SCL half period 4, 10, 25	Pass
`t_ps2`	Four keys with parity and stop bits at two clock rates	Pass
`t_ethernet`	Manchester line timed in nanoseconds with ±3 ns jitter and random phase at 50 and 60 MHz; all broadcast frames intact and CRC checked, other addresses rejected; a 64-byte frame reported as overflow	Pass
`t_manchester`	Manchester transmit at 6 and 10 clocks per bit, decoded by the other channel; end-of-frame idle and release	Pass
`t_usb`	Receive with stuffing at 33.33 clocks per bit; SE1 and missing stuff bit abort with line error; transmit decoded by a model, end of packet SE0 SE0 J then released; channel B transmit looped into channel A receive	Pass
`t_random`	1,800 random programs (6 seeds × 150, both bit orders): 0–6 headers, fields of 1–8 bits, capture on and off, random data, mismatches, splits across frames	Pass

Bugs found by these tests and fixed.

- The frame buffer's full check compared a 32-bit difference; after the pointers wrapped it never saw "full" and overwrote unread entries.
- Header and START pushes are registered, so write backpressure must stop at "almost full".
- Chip select glitched between a write frame completing and its last bits reaching the codec.
- STOP and the end of packet waited for the return link; they now follow the frame mark.
- Outputs drove low after reset; they are now off until `OUT` is given.

Lint. Verilator reports no errors; the remaining warnings are intentional width extensions. There are no combinational loops (Yosys `check -assert`).

7 Implementation results

Yosys and ABC with the IHP SG13G2 liberty files, before place and route.

Block	Cells	Area, typical (μm^2)
Frame chain	3,974	73,000
Line codec	1,114	14,400
Host FIFO (8 entries)	284	8,400
One channel (chain, codec, host FIFO, dispatcher)	5,388	96,100
Control unit	910	19,800
UART receiver with auto-baud	442	7,700
Return path with UART transmitter	317	5,300
Whole design (flattened)	**12,031**	**219,500**

Corner	Worst register-to-register path	Budget at 50 MHz
Typical, 1.2 V, 25 °C	4.1 ns	20 ns
Slow, 1.08 V, 125 °C	6.6 ns	20 ns

The worst path is inside MAIN: bit counter, length compare, `complete`, and the reset of the other blocks. The line codec's worst path is 3.4 ns at the slow corner. Figures move by a few cells from run to run with the order the files are read in. Allowing for clock-to-Q, setup and wiring after placement, there is roughly a factor of two in hand. The Tiny Tapeout flow's post-route STA is the sign-off check.

8 Limits and future work

1. **Return link bandwidth.** Two bytes per host entry. At 4 clocks per bit the link keeps up with USB low speed and the clocked standards. Full-size 10BASE-T frames (64 bytes and more) arrive faster; they are received but overflow the 16-entry buffer and are reported with the overflow flag.
2. **Transmit packet size.** The host FIFO holds 8 entries and the instruction link is slow, so a transmitted packet is what the write blocks hold plus 8 `DATA` entries (about 14 bytes).
3. **Head-of-line blocking.** A host entry waiting for the return link holds up the wire entries behind it. I^2C just stretches SCL; a timed transmit would pause.
4. **Manchester limits.** Receive needs at least 5 clocks per bit (50 MHz for 10BASE-T); transmit needs an even divider (40 or 60 MHz).
5. **Not supported yet.** I^2C target mode (needs a branch on the received R/W bit and a START/STOP detector); a read header that captures any value (for the USB PID); CRC generation on chip.
6. **Reset.** `CLEAR` resets a channel through its asynchronous reset (`rst_n && !clear`), driven from a control-unit flip-flop.
7. **Tiny Tapeout wrapper** and final pin map.

9 Files

Folder or file	Content
`rtl/top/`	`protocol_emulator.sv` (top level) and `protocol_emulator_inst.sv` (instantiation template)
`rtl/core/`	`emulator_channel`, `control_unit`, `line_codec`, `entry_dispatch`, `return_path`
`rtl/core/frame_chain/`	`frame_chain` and its blocks: `first_counter`, `header_counter`, `start_router`, `main_counter`, `end_counter`, `watchdog`
`rtl/primitives/`	Reusable building blocks: `uart_rx`, `uart_tx`, `host_fifo`, `frame_buffer`
`rtl/files.f`	File list in compile order, used by the scripts
`sim/`	Test bench `bench.h` and the tests `t_*.cpp`
`scripts/`	`run.sh` (every test, then synthesis), `sta.sh` (timing)
`docs/`	This document, the ISA reference, the architecture diagram and its generator

`bash scripts/run.sh` needs Yosys and g++. Setting `LIBERTY` to an IHP liberty file also reports area. Every source file starts with a header giving the file, module, type (top level, core RTL or primitive), place in the hierarchy and submodules. The RTL is SystemVerilog 2008 in the project's coding style, checked with Yosys and Verilator; it has not yet been compiled in Quartus.